Reviewer Pack — Minimal Hodge Structure Rank and Circuit Monotone Width Ineq…

Entry 1163cbf69f65 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 16:39:29 UTC

Minimal Hodge Structure Rank and Circuit Monotone Width Inequality

Entry ID: `1163cbf69f65` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 16:39:29 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every Boolean circuit C computing a function f: {0,1}^n -> {0,1}, the minimal Hodge structure rank (h(C)) of C is linearly related to its monotone width w_C, such that h(C) = Ω(w_C^(2/3)) and h(C) = O(√w_C).

Rationale (proposer's reasoning):

> Hodge structures capture complex geometric properties that might correlate with circuit complexity measures like monotone width. The minimal rank of the Hodge structure could potentially reveal non-trivial structural information about the circuit, offering a novel perspective on circuit complexity.

Taxonomy category: `Algebraic Geometry × Boolean Circuit Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7c62336f10a006d9`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, after analyzing at least 40 circuits, the Pearson correlation coefficient between the minimal Hodge structure ranks h(C) and the monotone widths w_C is within the bounds [√(w_C^(2/3)), √w_C]. It is falsified if any seed produces a correlation coefficient outside these bounds or if less than 40 circuits are analyzed.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge structure rank" AND "circuit monotone width" - "minimal Hodge structure rank" AND circuit complexity - "Boolean circuit" AND Hodge theory AND monotone width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def monotone_width(circuit):
        n = len(circuit)
        width = 0
        for i in range(n):
            if circuit[i] == 1:
                width += 1
        return width

    def hodge_structure_rank(circuit):
        n = len(circuit)
        rank = 0
        for i in range(n):
            if circuit[i] == 1:
                rank += 1
        return rank

    instances_tested = 0
    total_h = 0
    total_w = 0
    max_n = 0

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(6):
```

```
            circuit = generate_circuit(n)
            w = monotone_width(circuit)
            h = hodge_structure_rank(circuit)
            total_h += h
            total_w += w
            instances_tested += 1
            max_n = max(max_n, n)

    mean_h = Fraction(total_h, instances_tested)
    mean_w = Fraction(total_w, instances_tested)
    correlation_coefficient = (mean_h * mean_w) / (mean_h**2 + mean_w**2)

    conjecture_holds = math.sqrt(mean_w**(2/3)) <= correlation_coefficient <= math.sqrt(mean_w)
    counterexample = "" if conjecture_holds else f"Correlation coefficient {correlation_coefficien

    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": float(correlation_coefficient),
        "instances_tested": instances_tested,
        "n_max": max_n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE insufficient support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means it did not complete the analysis of at least 40 circuits as required by the pre-registered criterion. | next: Re-run the test with a longer timeout or an alternative method to ensure that the analysis of at least 40 circuits is completed.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14179
2	preregistration	ollama_remote	glm4:latest	0	0	10930
3	novelty	ollama_remote	glm4:latest	0	0	8380
4	novelty	ollama_remote	glm4:latest	0	0	8670
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14568
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11513
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11470
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8842
9	judge	ollama_remote	glm4:latest	0	0	41062

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 129615 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1163cbf69f65.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1163cbf69f65.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1163cbf69f65.tar.gz` (if generated)